

Metrikagyűjtés Media over QUIC hálózatokban

Önálló laboratórium beszámoló

Készítette:	Konzulensek:
Horváth Benedek	Németh Felicián
D86EP7	Lévai Tamás

2025/2026. tanév, I. félév



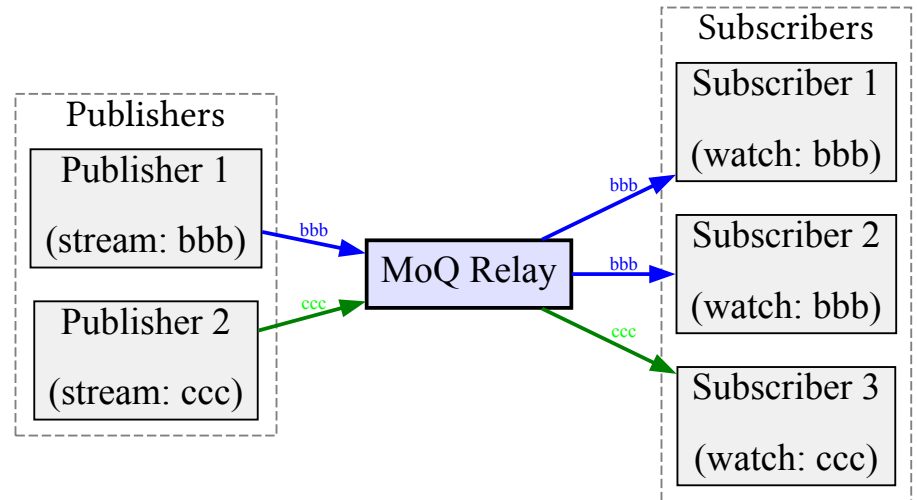
A Media over QUIC (MoQ) ökoszisztéma

Az architektúra három fő szereplőre épül:

- **Publisher:** Forrás
- **Relay:** Közvetítő, puffer
- **Subscriber:** Fogyasztó

A MoQ-adatmodell:

- **Namespace:** Streamek csoportja
- **Stream:** Trackek csoportja
- **Track:** Sáv, adatfolyam (pl. videó)
- **Group / Object:** Átviteli egységek (pl. videódarab)



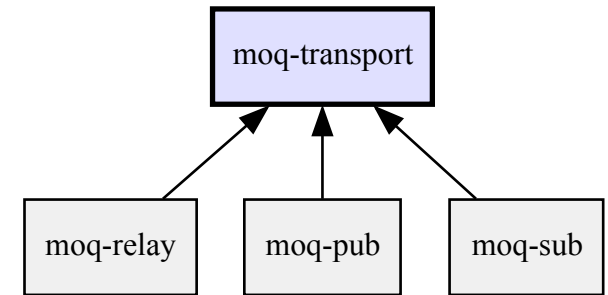
Egy egyszerűsített MoQ-hálózat

Kiindulási alap

A mérések alapjául szolgáló technológiák és implementációk:

QUIC Protokoll:

- UDP-alapú szállítási réteg
- Beépített titkosítás (TLS 1.3)
- TCP helyett



moq-rs komponensek egy részlete

moq-rs:

- A Media over QUIC (MoQ) specifikáció **Rust** nyelvű implementációja
- Jelenleg a Cloudflare kezelésében

A probléma

A moq - rs egy komplex, elosztott rendszer, rengeteg független komponenssel.

Miért nehéz leírni a hálózat minőségét?

- **Elosztott működés:** Több független komponens kommunikál
- **Változó hálózat:** A kapcsolatok és streamek folyamatosan változnak
- **Titkosítás:** A QUIC forgalom titkosított, nehéz „belehallgatni”

Erre lehet megoldás egy metrikagyűjtési megoldás implementálása.

Miért mérjük?

Szeretnénk a felhasználónak a lehető legjobb élményt nyújtani.

- **Felhasználói élmény javítása:** Akadozásmentes lejátszás, alacsony késleltetés
- **Erőforrás-gazdálkodás optimalizálása:** Terheléselosztás, hirtelen kiugrások észlelése
- **Hibakeresés:** Hol vesznek el a csomagok? Van-e memóriaszivárgás?

A fentiekhez szükséges adatokat mérésekkel kaphatjuk meg.

A kiválasztott metrikák

A fenti célok eléréséhez a gyűjtött adatokat így kategorizálhatjuk:

- **Hálózati metrikák (QUIC-ből)**
 - RTT, csomagvesztés
 - Pl. ha magas az RTT, akkor késik az adás
- **Forgalmi metrikák (MoQ-hálózat)**
 - Objektumok száma, aktív feliratkozók
 - Pl. a folyamatok népszerűsége alapján lehet optimalizálni a terhelésselosztást
- **Rendszermetrikák**
 - CPU használat, Memóriafoglalás
 - Pl. folyamatosan növvő **memória**használat szivárgásra utal.

Tervezés

Hogyan gyűjtsük az adatokat?

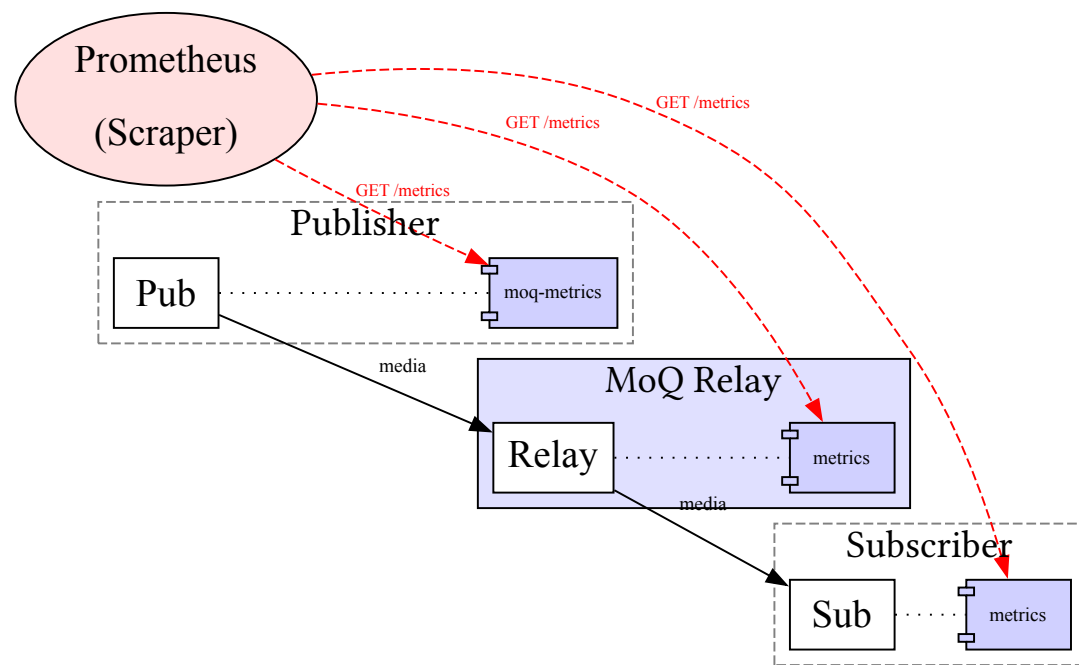
Prometheus:

- Pull modell
- Széleskörű használat
- Egyszerű integráció (HTTP endpoint)
- Kisebb erőforrásigény
- Egyszerűbb használat

Alternatíva: OpenTelemetry (push modell) - rugalmasabb backend szempontból (pl. Prometheus is lehet), de komplexebb

Architektúra

- **moq-metrics komponens:**
Különálló, lazán illeszkedő könyvtár.
- A moq-rs-en belül bármelyik komponens egyszerűen hozzáadhatja
- Minden komponensen HTTP szervert indít a /metrics végponton
- A Prometheus erről gyűjti be az adatokat



A mérőrendszer integrációja

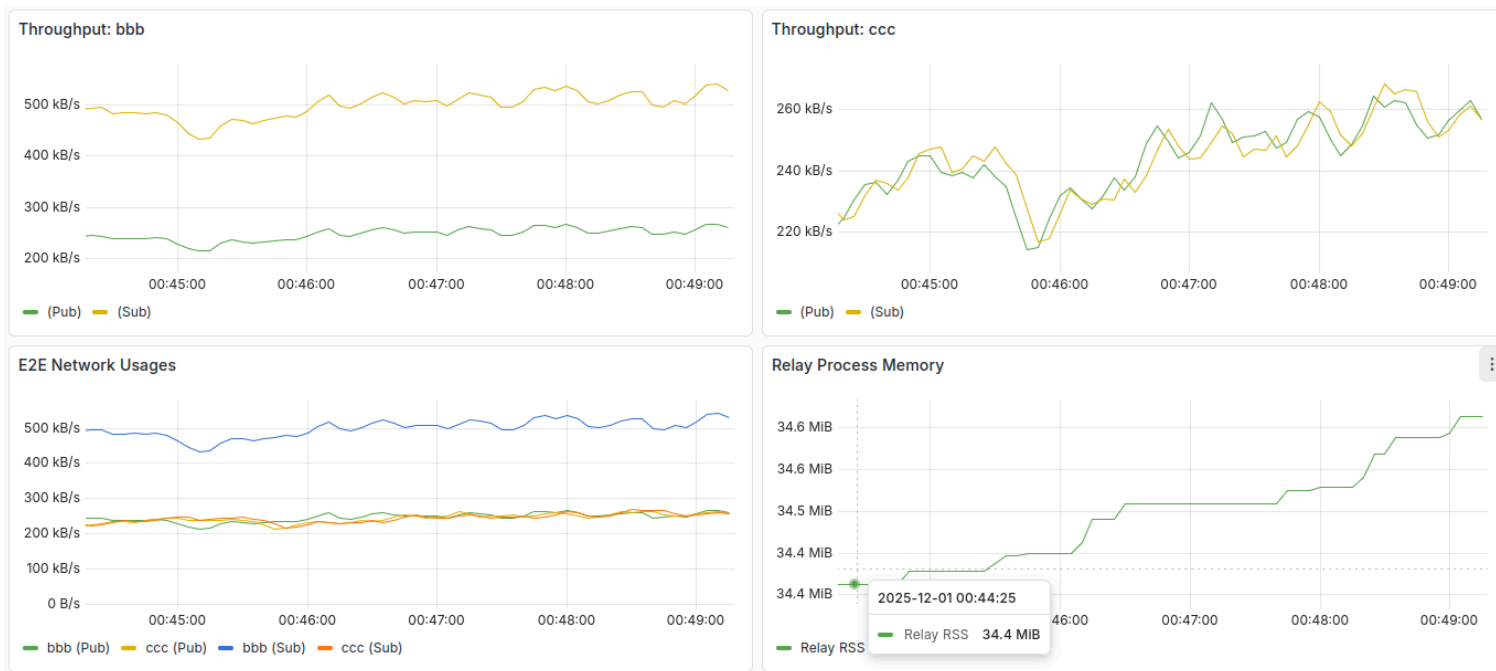
Implementáció

- Rust nyelv
- Globális hozzáférés (GLOBAL_METRICS) Singleton segítségével
- automatikus erőforrás-felszabadítás és számlálócsökkentés (RAII-elv)
- Rust makrók `define_metrics!` a boilerplate kód csökkentésére

```
define_metrics! {  
    // ...  
    objects_sent:  
        Counter<u64>,  
        "moq_relay_objects_sent",  
        "Total objects sent",  
    // ...  
}
```

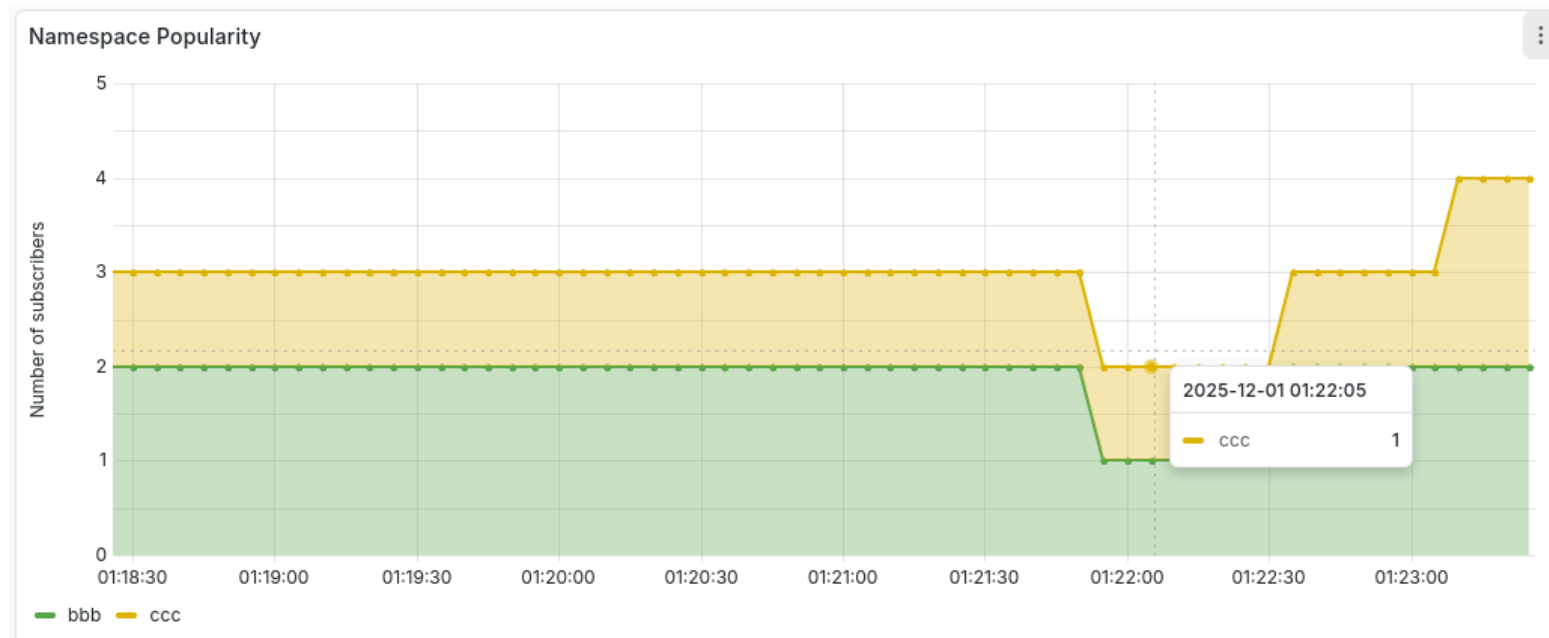
Egy metrika definíciója makróval

Kiértékelés



A megvalósított Grafana dashboard egy részlete

Kiértékelés



A különböző streamek feliratkozóinak eloszlása a hálózatban

Összegzés

- Lazán illeszkedő metrikagyűjtési könyvtár a moq - rs-hez
- Prometheus és Grafana használata
- Valós idejű betekintés a rendszer működésébe

Jövőbeli lehetőségek:

- Minimális PR az eredeti moq - rs projektbe
- E2E késleltetés mérése időbélyegek hozzáfűzésével
- További metrikák (pl. cachelés mérése, puffertelítettség).

Összegzés

- Lazán illeszkedő metrikagyűjtési könyvtár a moq - rs-hez
- Prometheus és Grafana használata
- Valós idejű betekintés a rendszer működésébe

Jövőbeli lehetőségek:

- Minimális PR az eredeti moq - rs projektbe
- E2E késleltetés mérése időbélyegek hozzáfűzésével
- További metrikák (pl. cachelés mérése, puffertelítettség).

Köszönöm a figyelmet!